

Distributed Training Systems

Working notes from implementing DDP, FSDP2, 3D parallelism, MoE, FP8 training, and fault tolerance from scratch — the mechanisms behind how models past a single GPU's memory actually get trained. Each section pairs the standard reference (paper, diagram) with a from-scratch implementation checked against a single-device ground truth.

M4 MacBook · PyTorch 2.13 · gloo backend (CPU-simulated collectives, same semantics as NCCL on GPU)

§1

1. Foundations – a production training loop

Mixed precision, gradient accumulation, checkpointing, early stopping.

A minimal GPT (causal self-attention, pre-norm residual blocks, weight-tied embeddings) written without importing nanoGPT or HuggingFace, so every line is something to reason about. The part that matters for this course isn't the model — it's the training loop around it, which needs a few properties toy scripts skip:

- **bf16 autocast**, not fp16 — bf16 keeps fp32's exponent range (8 bits), just fewer mantissa bits, so it needs no loss scaling. fp16's narrow exponent range underflows typical gradient magnitudes without it.
- **Gradient accumulation** — simulate a larger batch than fits in memory by summing several micro-batch backward passes before one optimizer step.
- **Checkpoint / resume with zero loss discontinuity** — the property most worth testing in any training stack, and the easiest to get subtly wrong.

```
# standard practice (GPT-3 paper, Appendix B): decay weight matrices,
# not LayerNorm/bias/embeddings – the split that matters for AdamW
def build_optimizer(named_parameters, weight_decay, lr, betas):
    decay, no_decay = [], []
    for _, p in named_parameters:
        (decay if p.dim() >= 2 else no_decay).append(p)
    groups = [{"params": decay, "weight_decay": weight_decay},
              {"params": no_decay, "weight_decay": 0.0}]
    return torch.optim.AdamW(groups, lr=lr, betas=betas)
```

Trained on tiny-Shakespeare (304K tokens), the model overfit exactly the way a 30M-parameter model on a 300K-token corpus should: train loss fell to 0.44, but val loss bottomed at step 500 (4.72) and rose for the rest of the 3000-step run.

WHAT RUNNING IT CAUGHT

The checkpoint-pruning policy ("keep the last 3") had already deleted the step-500 checkpoint by the time training finished — every checkpoint left on disk was worse than the one that actually generalized. *"Can I resume after a crash"* and *"which checkpoint should I deploy"* are different questions; conflating them silently discards the one you want. Fixed with an independently-tracked `best.pt` (pinned by eval loss, immune to pruning) plus early stopping — the rerun auto-stopped at step 1500, half the compute, with the right checkpoint pinned.

§2

2. Data parallelism – DDP

Every rank holds the full model. Only the gradients need to agree.

DistributedDataParallel replicates the entire model on every device. Each rank runs forward/backward on a different slice of data, producing its own gradients; a collective **all-reduce** then averages those gradients across every rank before the optimizer step, so all replicas apply an identical update and stay bit-for-bit in sync. The effect is mathematically close to training with an $N\times$ larger batch, at $N\times$ the throughput.

The PyTorch DDP paper (Li et al., [2020](#)) adds two optimizations worth knowing: **gradient bucketing** (batch many small tensors into one collective call instead of thousands of tiny ones — fixed per-call overhead otherwise dominates) and **overlap with backward** (all-reduce a bucket the moment its gradients are ready, while backward keeps computing earlier layers — hides communication behind compute that's happening anyway).

Grad accumulation needs an explicit sync boundary

DDP's default is to all-reduce on every `backward()` call — wasteful when several micro-batches are meant to accumulate locally before one real step. `no_sync()` defers that:

```
for micro_step in range(grad_accum_steps):
    x, y = get_batch(...)
```

```
is_last = micro_step == grad_accum_steps - 1
# only the LAST micro-step's backward() triggers the real all-reduce
sync_ctx = model.no_sync() if (is_ddp and not is_last) else nullcontext()
with sync_ctx:
    loss = model(x, y) / grad_accum_steps
    loss.backward()
```

WHAT RUNNING IT CAUGHT

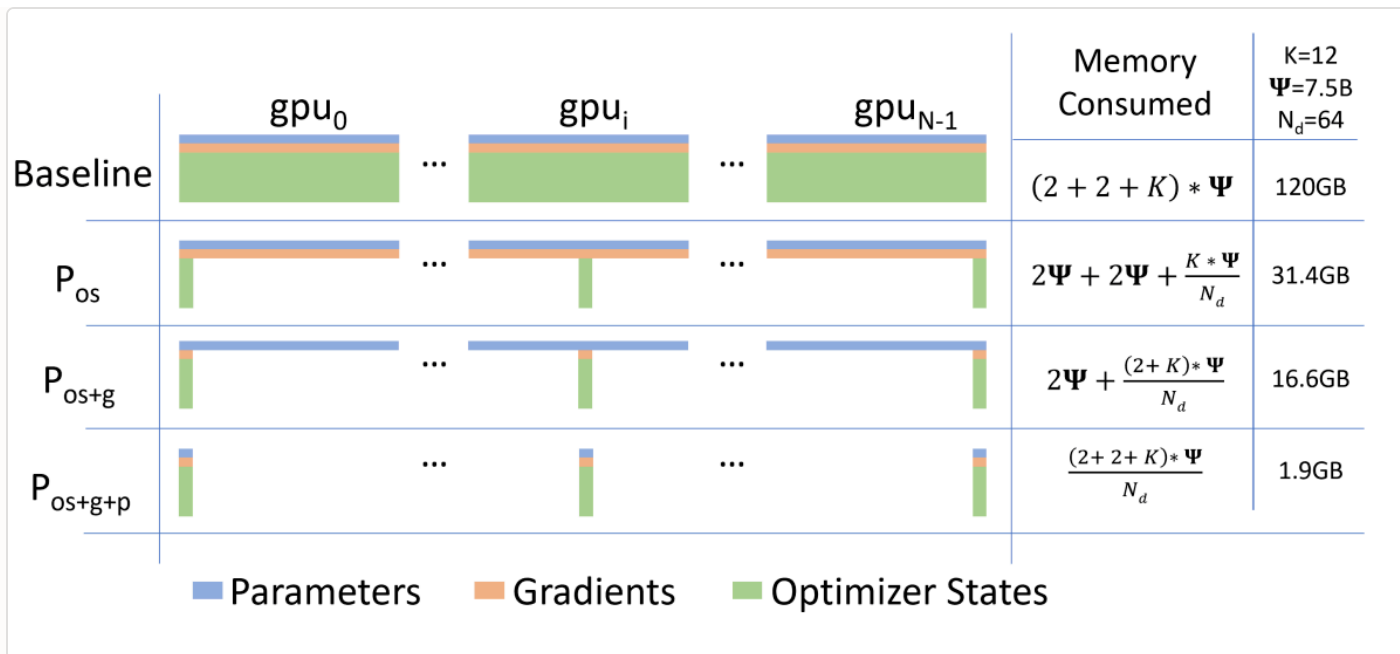
macOS's default `torchrun` rendezvous address (`localhost`) triggers a reverse-DNS lookup for `::1` that hangs and retries for minutes without a configured PTR record. `--master_addr=127.0.0.1` — an IP literal — skips the lookup. Verified with 4 real processes over gloo: `world_size=4` confirmed, and only rank 0 emitted log/checkpoint output, confirming the `is_master` gating holds under real concurrency (it was only ever exercised single-process before).

§3

3. FSDP2 – sharding everything

No rank ever holds the full model. ZeRO stage 3, PyTorch-native.

DDP's ceiling is memory: every rank stores a full copy of parameters, gradients, and optimizer state. **ZeRO** (Rajbhandari et al., [2019](#)) shards these across ranks instead, in three cumulative stages — optimizer state, then gradients, then parameters — trading more communication for less per-device memory. FSDP is stage 3 (`Pos+g+p`): full sharding.



Per-GPU memory at each ZeRO stage, for a 7.5B-parameter model across 64 GPUs (Ψ=7.5B, N_d=64, mixed-precision Adam K=12). Baseline 120GB → P_{os} 31.4GB → P_{os+g} 16.6GB → P_{os+g+p} 1.9GB. [Rajbhandari et al. 2019, Fig. 1](#)

The same math scaled to this course's own numbers: mixed-precision Adam costs 16 bytes/parameter (4B weight + 4B grad + 4B momentum + 4B variance, all fp32 here — no separate bf16 master copy). A 30M-param model needs 480MB total; under FSDP with 4 ranks, each holds exactly 120MB. At 100B parameters: **1.6TB** — doesn't fit on an 80GB H100 under DDP *at all*; under FSDP with 64 GPUs, 25GB/GPU. Past a certain scale this isn't an optimization, it's the only way training fits on the hardware.

DTensor: sharded storage, unchanged shape

FSDP2 (`torch.distributed.fsdp.fully_shard`) represents sharded parameters as **DTensor** — a tensor subclass whose `.shape` reports the full logical shape while only a shard is physically stored, with a `Shard(dim)` placement recorded against a device mesh. Confirmed directly on this course's own model:

```
blocks.0.mlp.fc_in.weight: logical_shape=(1536, 384)
  placements=(Shard(dim=0),) LOCAL shard shape=(384, 384)
# 4 ranks, each holding exactly 25.0% of parameters — measured, not assumed
```

This is why code like `build_optimizer`'s `p.dim() >= 2` check above needs no special-casing under FSDP2: DTensor reports the pre-shard shape. The older wrapper-style FSDP flattened everything into an opaque 1-D `FlatParameter` per unit, losing that structure — DTensor is the reason FSDP2 exists as a separate API.

THREE PROPERTIES DDP NEVER HAS TO WORRY ABOUT

Optimizer must be built after wrapping — sharding replaces the underlying storage, so an optimizer built from pre-wrap parameter references would silently optimize tensors that no longer back the model. **Forward is itself a collective** — every block all-gathers its full parameters before computing, so anything that calls forward (including eval) must run on every rank, not just rank 0, or the others deadlock waiting at an all-gather nobody joins.

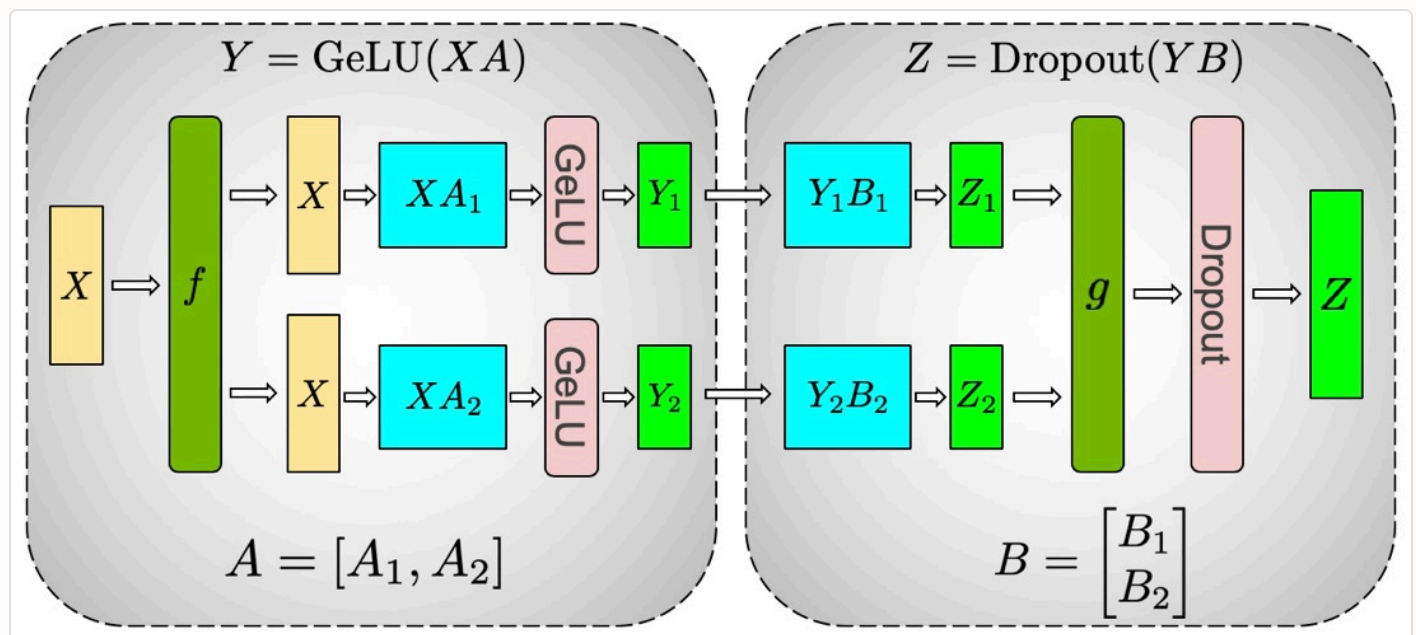
Checkpoint gathering is collective too — reassembling a full state dict from shards needs every rank's participation even though only rank 0 writes to disk. All three verified end to end: training, eval, checkpoint save, and resume (including optimizer state) round-tripped correctly over 4 real processes.

§4.1 — 3D PARALLELISM

4.1 Tensor parallelism

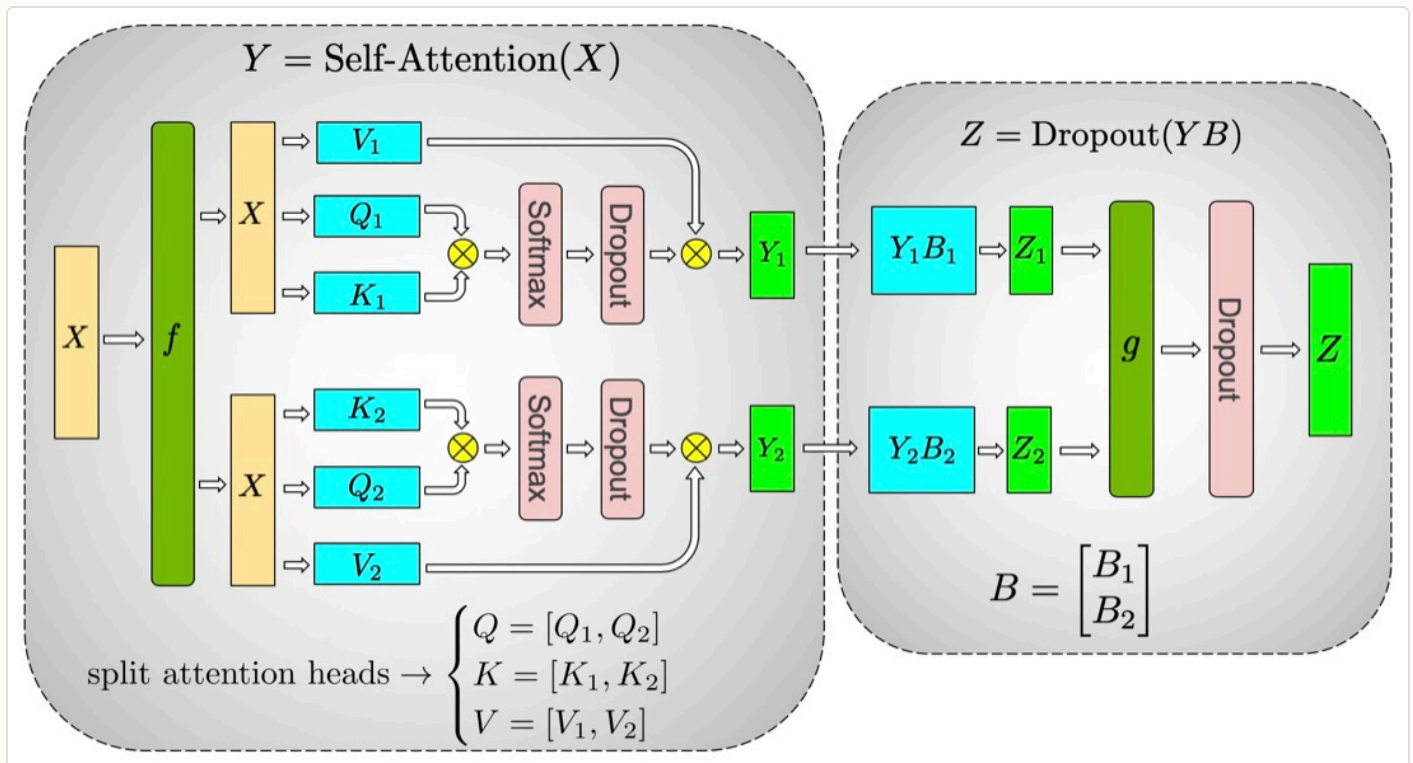
Split one weight matrix across devices, not the batch or the layers.

Megatron-LM (Shoeybi et al., 2019) splits a single Linear layer's weight matrix directly. For an MLP block (`fc_in: d→4d`, GELU, `fc_out: 4d→d`), the trick is pairing a **column split** on the first layer with a **row split** on the second:



f: identity forward, all-reduce backward. **g**: all-reduce forward, identity backward. Splitting A by columns needs no forward communication (GeLU applies independently to each column-slice); splitting B by rows needs exactly one all-reduce to sum the partial outputs. [Shoeybi et al. 2019, Fig. 3](#)

Attention uses the same pattern: QKV projection is column-parallel (split by head — each rank computes complete attention for its own heads, no cross-rank communication needed for the attention math itself), and the output projection is row-parallel.



Same f/g pattern applied to self-attention — QKV split by head (column-parallel), output projection row-parallel.

[Shoeybi et al. 2019, Fig. 3](#)

Net result: exactly **2 all-reduces per transformer block** (one after attention, one after the MLP) in the forward pass, and 2 more in backward — not one per linear layer.

```
class _ToParallelInput(torch.autograd.Function):
    """f: identity fwd, all-reduce bwd. Input is already replicated, so no
    forward comm — but each rank's local backward only computes a PARTIAL
    dL/d(input); the true gradient needs summing across ranks."""
    def forward(ctx, x): return x
    def backward(ctx, grad_output):
        dist.all_reduce(grad_output, op=dist.ReduceOp.SUM)
        return grad_output

class _FromParallelOutput(torch.autograd.Function):
    """g: all-reduce fwd, identity bwd — the mirror image."""
    def forward(ctx, x):
        dist.all_reduce(x, op=dist.ReduceOp.SUM)
        return x
    def backward(ctx, grad_output): return grad_output
```

CHECK	MAX DIFF VS. SINGLE-DEVICE REFERENCE
forward output	1.49e-08
grad w.r.t. input	9.31e-09
grad w.r.t. fc_in weight	0.00e+00
grad w.r.t. fc_out weight	0.00e+00

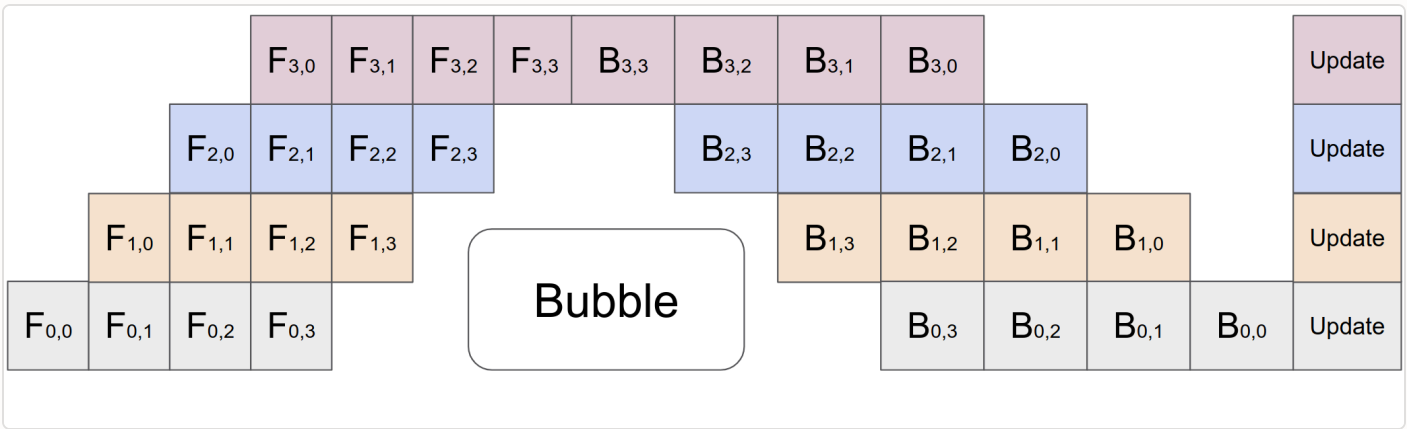
Differences at float32 noise floor — this is the correctness bar that matters, since at real scale there's never a spare single device to check a production run against.

§4.2 — 3D PARALLELISM

4.2 Pipeline parallelism

Split by layer, across devices — and the microbatching that makes it not a waste of hardware.

Pipeline parallelism assigns different *layers* to different devices — device 0 holds layers 1–2, device 1 holds layers 3–4, and so on. Naively, only one device is ever active: while device 0 computes, devices 1–3 are idle, then the reverse. **Microbatching** (GPipe, Huang et al., [2018](#)) splits each batch into M pieces and pipelines them through the stages like an assembly line, so stage 2 can process microbatch 3 while stage 1 is already on microbatch 4.



4 devices, GPipe's schedule: forward passes (F) fill the pipe, backward passes (B) drain it. The gap in the middle — every device waiting for the first microbatch to reach the last stage and start flowing back — is the **bubble**. [Huang et al. 2018, Fig. 2](#)

For P stages and M microbatches, the bubble fraction is $(P-1)/(M+P-1)$ — shrinks as M grows relative to P , but never reaches zero (the first and last microbatch always pay the full pipeline fill/drain cost).

1F1B: same bubble, less memory

GPipe's schedule runs *all* forwards before *any* backward starts, so it must hold all M microbatches' activations in memory simultaneously. PipeDream's **1F1B** ("one-forward-one-backward") interleaves instead, starting backward as soon as possible — peak activations in flight drops to roughly P , not M . A common misconception is that 1F1B is *faster*. Measured directly, using PyTorch's real `torch.distributed.pipelineing` (`ScheduleGPipe` / `Schedule1F1B`, the same library TorchTitan uses):

SCHEDULE	MICROBATCHES	WALL TIME	GRAD VS. REFERENCE
GPipe	8	5.19ms	$\sim 1e-9$
1F1B	8	4.46ms	$\sim 1e-9$

Nearly identical speed at the same microbatch count — exactly what sharing a bubble formula predicts. 1F1B's real advantage is memory, not throughput.

WHAT RUNNING IT CAUGHT

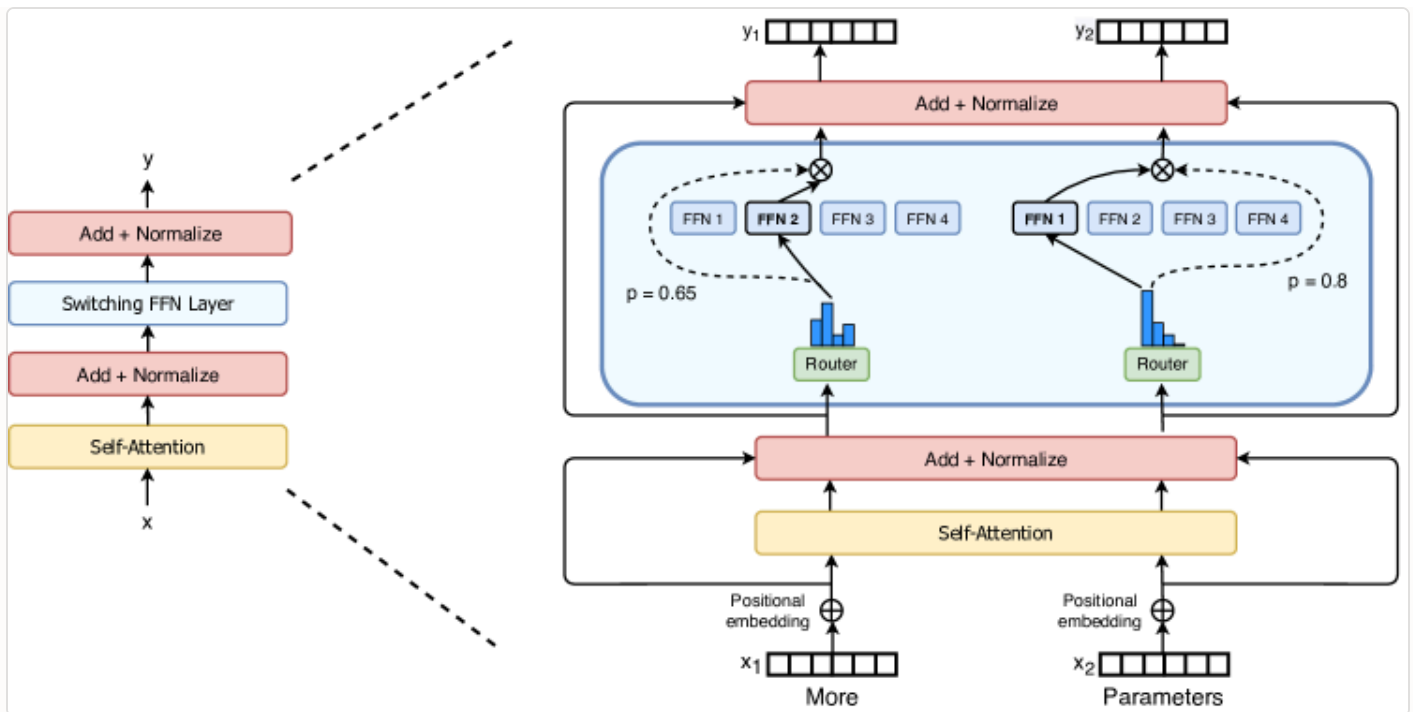
`Schedule1F1B` raises `ValueError` if microbatches < stages ("must be $\geq$ number of stages") — it needs enough microbatches in flight to interleave. `ScheduleGPipe` has no such floor; it just gets bubblier. A real, load-bearing difference in the two schedules' constraints, not documented anywhere obvious.

§4.3 — 3D PARALLELISM

4.3 Mixture of Experts & expert parallelism

How 100B+ models get built cheaply — and why their communication pattern is fundamentally different.

A MoE layer replaces one dense FFN with N narrow "expert" FFNs and a small router that picks the top- k experts per token. Total capacity scales with N ; compute per token stays roughly fixed (only k experts run per token, not N).



Top-1 routing (Switch Transformer): the router produces a probability over experts per token, the highest-probability expert processes it, output scaled by that probability. [Fedus et al. 2021, Fig. 2](#)

Load balancing without touching the loss

Classic MoE (Switch Transformer, GShard) adds a load-balancing term to the training loss, which can fight the primary objective. DeepSeek-V3's **auxiliary-loss-free** approach steers routing with a per-expert bias that never enters the loss at all:

```
# a BUFFER, not a Parameter – outside the autograd graph entirely
self.register_buffer("expert_bias", torch.zeros(n_experts))

# bias affects which experts get PICKED (topk selection); the gate WEIGHT
# applied to a picked expert's output still comes from the unbiased score
biased_scores = scores + self.expert_bias
_, topk_idx = biased_scores.topk(self.top_k, dim=-1)

# after each step: overloaded experts' bias moves down, underloaded up –
# fixed step size, no gradient involved at all
target = tokens_per_expert.mean()
self.expert_bias += bias_update_rate * torch.sign(target - tokens_per_expert)
```

experiments/moe_layer.py – verified: most-loaded expert's bias moved to -0.001, least-loaded to +0.001

Expert parallelism: data-dependent communication

At real scale, experts are distributed across ranks (DeepSeek-V3: 256 routed experts over 64 GPUs). Tokens are dispatched to whichever rank holds their selected expert via **all-to-all**, computed there, and results all-to-all'd back. Unlike tensor or pipeline parallelism — where the communication shape is fixed and known ahead of time — MoE's dispatch volume depends on what the router decides *this batch*:

```
[rank 0] tokens sent to each rank: [2, 3, 3, 4]    received: [2, 4, 5, 3]
[rank 1] tokens sent to each rank: [4, 4, 2, 2]    received: [3, 4, 2, 6]
max|diff| vs single-device reference: 5.96e-08
```

experiments/expert_parallel_demo.py – 4 ranks, every rank sends/receives a different amount

This is why production dispatch kernels (DeepSeek's DeepEP) need a "how much am I about to receive from you" round before the actual payload — the shape isn't known statically.



DualPipe: microbatches fed from both ends of the pipeline simultaneously, so one chunk's expensive cross-node all-to-all overlaps another chunk's actual compute — built specifically because MoE's dispatch/combine has a much worse compute:communication ratio than dense pipeline parallelism. [DeepSeek-AI 2024, Fig. 5](#)

WHAT RUNNING IT CAUGHT

The list-based `dist.all_to_all` doesn't reliably support ragged (different-size-per-rank) tensors on the gloo backend — errors the moment two ranks send different amounts.

`all_to_all_single` with explicit `input_split_sizes` / `output_split_sizes` handles it correctly — the pattern real dispatch kernels are built around.

§5

5. FP8 quantization

A single-device numerics problem — no distributed anything, and an honest negative result.

DeepSeek-V3 is the first large, openly-documented model trained substantially in FP8 — for real speed and memory, not just quantized inference. E4M3 (1 sign + 4 exponent + 3 mantissa bits) represents values up to ± 448 ; E5M2 trades mantissa bits for a wider range (± 57344), typically used for gradients since they have wider dynamic range than weights.

```
x          = [0.001,  0.02,   1.0,  100.0,  448.0,  500.0]
e4m3(x)    = [0.00195, 0.0195, 1.0,   96.0,  448.0,  448.0] # clipped: 500 > max
```

The dynamic-range problem – and why the naïve intuition about it is wrong

A single scale factor for a whole tensor lets a few outlier values (the well-documented "outlier feature" phenomenon in transformers — Dettmers et al., [2022](#)) force a large scale on every other value. Intuition says this should crush "typical" values' precision. Measured directly, it doesn't — much:

SCALING	TYPICAL-VALUE REL. ERROR	OUTLIER REL. ERROR
naive per-tensor	0.0226	0.0224
block-wise (n=32)	0.0220	0.0027

FP8 is still **floating point**, not fixed point — relative precision stays roughly constant across its representable range regardless of scale, unless a value is pushed toward **underflow**. At a 40× outlier ratio, typical values land nowhere near FP8's underflow floor, so per-tensor scaling barely hurts them. Block-wise scaling *does* measurably fix the outliers' own error (8×) — each gets a scale sized to its own block instead of being stretched by unrelated outliers elsewhere in the tensor.

```
class _QuantizeDequantizeSTE(torch.autograd.Function):
    """Straight-through estimator (Bengio et al. 2013). Chaining the raw
    (x/scale).to(fp8).to(fp32)*scale composition directly and calling
    .backward() gives an undocumented, SCALE-DEPENDENT gradient – measured
    1.95 at scale=0.001 vs. the correct 1.0 at scale=1.0. Real QAT never
    leans on whatever a cast op's autograd formula happens to do."""
    def forward(ctx, x, scale):
        return (x / scale).to(torch.float8_e4m3fn).to(torch.float32) * scale
    def backward(ctx, grad_output):
        return grad_output, None # identity – as if quantization hadn't happened
```

experiments/fp8_quantization.py

With the STE fix in place, full-precision, naive-FP8, and block-wise-FP8 trained a toy model to nearly identical loss curves. Not the dramatic confirmation expected going in — and reasoning through why is the actual finding: it points at DeepSeek-V3's documented pain point being the H800's **14-bit FP8 accumulation** precision (fixed by promoting to CUDA-core accumulation every 128 elements), not pure quantization-scale granularity. That accumulator behavior is structurally unreproducible on CPU, which always accumulates matmuls in full fp32/fp64 internally — a real boundary of what this kind of experiment can show.

§6

6. Reliability & control plane

The part standard tutorials skip — and the one that matches "research infrastructure" as a job description.

Data plane: the GPUs actually computing. **Control plane:** the layer deciding who runs what and what happens on failure. A useful taxonomy for "recovers from failure," in increasing sophistication:

1. **Level 0** — no automation; a human notices and restarts manually.
2. **Level 1** — an external supervisor (Slurm, k8s, `torchrun --max-restarts`) detects the crash and restarts the *whole worker group*, relying on checkpoint/resume.
3. **Level 2** — `torch.distributed.elastic`'s true elastic mode: membership can shrink/grow via rendezvous without necessarily killing everyone.
4. **Level 3** — torchft's per-step fault tolerance: surviving replicas never pause. The only one that's genuinely *self-healing* — making progress *through* a failure, not just recovering *after* one.

Level 1, verified live

Launched 4-process training, then `kill -9` 'd one worker mid-run to simulate a hard hardware failure:

```
13:17:14 kill -9 rank 2 (pid 34464)
13:17:14 elastic agent detects it, SIGTERMs all 3 surviving workers
```

```
13:17:15 4 fresh processes spawned, world_size=4 again
13:17:15 "Resumed from step 100" – automatic, zero manual intervention
```

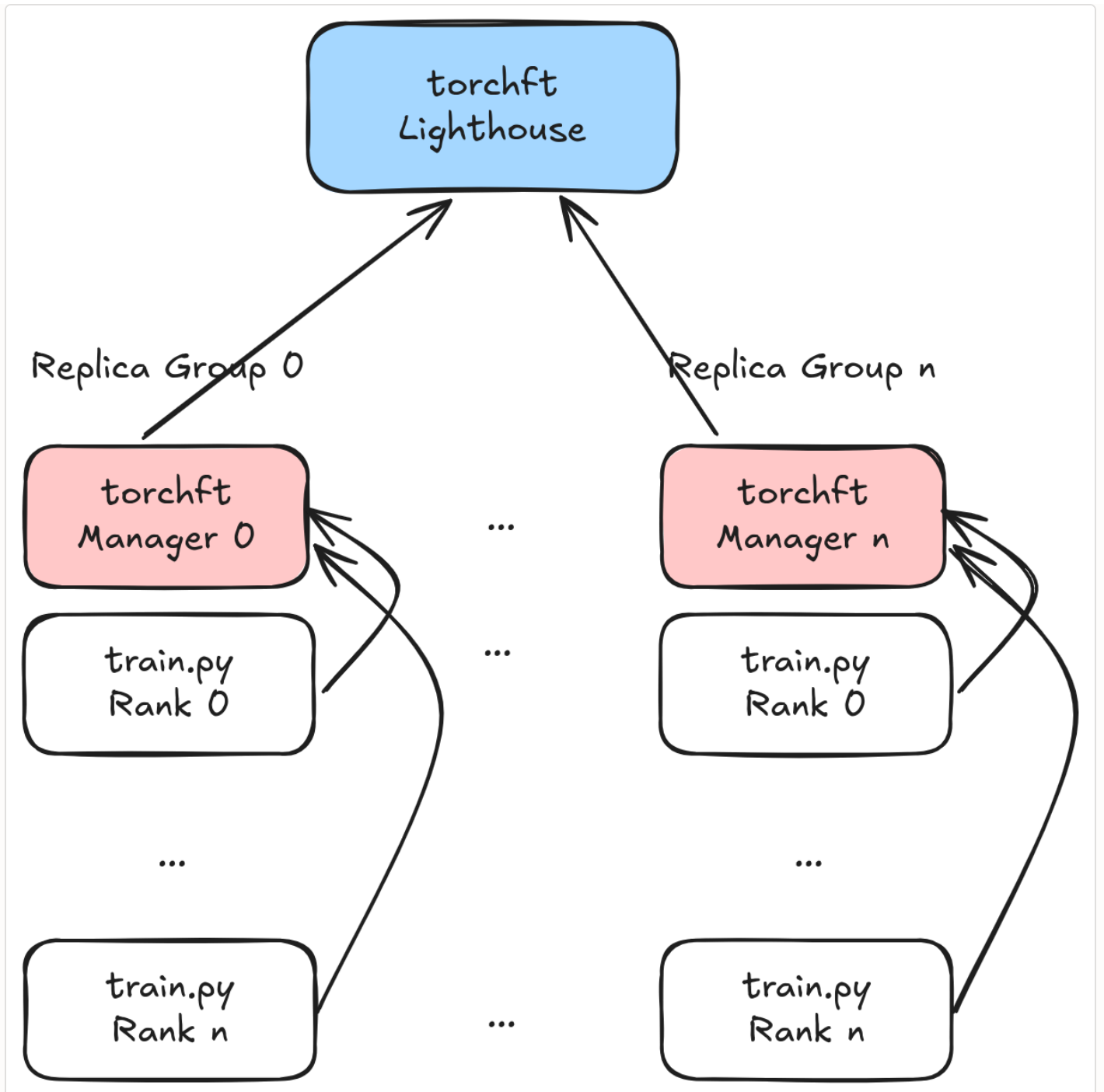
This is the literal mechanism behind Meta's report that Llama 3 405B training had 419 unexpected interruptions over 54 days and kept going anyway — this loop, running automatically, over and over.

WHAT RUNNING IT CAUGHT

The resumed run's loss at step 110 (8.0846) differed from the original run's step-110 loss (8.1602) — small, but real. Cause: this course's own notes had claimed checkpoints saved RNG state; they never did. `seed_everything` reseeds fresh from `seed+rank` on every process start rather than resuming a saved generator state, so a resumed run samples a different batch sequence than an uninterrupted one would have — no loss-curve discontinuity (the property actually tested and real), but not bit-exact data-order reproducibility (claimed, never tested). Corrected rather than quietly edited away.

torchft – per-step fault tolerance

A standalone Rust binary, `torchft_lighthouse`, coordinates across independent **replica groups** — not ranks within one `torchrun` call, but entire separate `torchrun` invocations (each potentially on different physical machines), every one pointed at the shared Lighthouse via `TORCHFT_LIGHTHOUSE=http://host:port`. Each replica group runs its own `Manager`, which sits between the training loop and the Lighthouse:



One Lighthouse, N replica groups. Each replica group is a full, independent `torchrun` job; its `Manager` is the only thing that talks to the Lighthouse — individual ranks never do. [torchft repo, README diagram](#)

The per-step protocol — this is the actual sequence, not a simplification of it:

replica 0
rank 0

replica 1
rank 0

training
step=100

initializing
step=0

Lighthouse quorum
exchange addresses and steps
 $\max(\text{step}) == 100$

if quorum changed:
Reinitialize FT ProcessGroup

forward()

forward()

heal()

request weights

backward()

backward()

zero_grad()

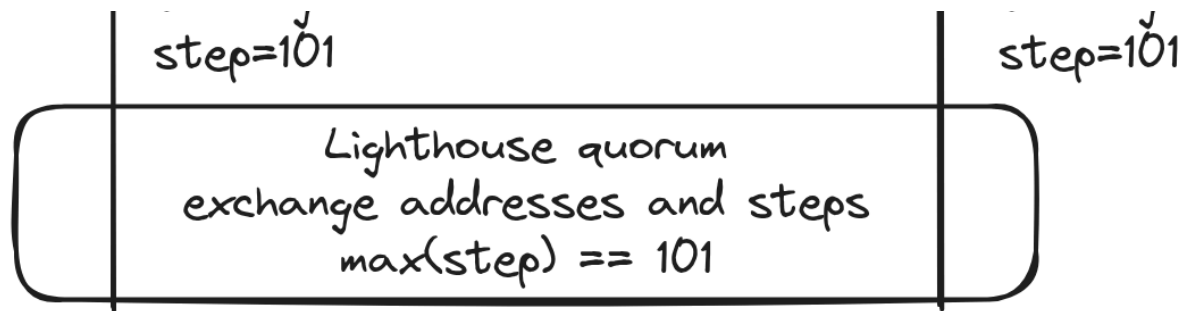
Fault Tolerant all_reduce(grad)

if should_commit():
optim.step()

if should_commit():
optim.step()

training

training



Replica 0 is mid-training at step 100; replica 1 is just joining at step 0. Both hit a Lighthouse quorum barrier that exchanges addresses and steps, computing `max(step)` — the joining replica's `heal()` then pulls current weights live from a healthy peer (replica 0), not from disk. Both then run forward/ backward, a fault-tolerant `all_reduce(grad)`, and gate the optimizer step behind `should_commit()` — which returns false if the quorum changed mid-step, discarding that step's update rather than applying a partial/inconsistent one. [torchft repo](#), [README diagram](#)

`should_commit()` is the detail worth remembering: fault tolerance isn't just "catch the exception and retry" — a step whose quorum changed *during* the all-reduce could apply an update computed from an inconsistent set of replicas. Gating the commit, not just the collective, is what keeps the training mathematically well-defined through a membership change. Recovery (`heal()`) pulling weights from a live peer rather than disk is the other detail: skips the I/O bottleneck that dominates recovery time for large models. Training code barely changes for any of this — `Manager`, `DistributedDataParallel`, and `Optimizer` wrapper classes absorb nearly all of it; a fault-tolerant loop reads almost identically to a normal DDP one.

Pathways – a different axis entirely

Not competing with torchft; solving a different problem, one level lower in the stack. Every system above — DDP, FSDP, torchft — is **multi-controller**: each worker runs an independent copy of the same client program (SPMD), coordinating only through explicit collectives.

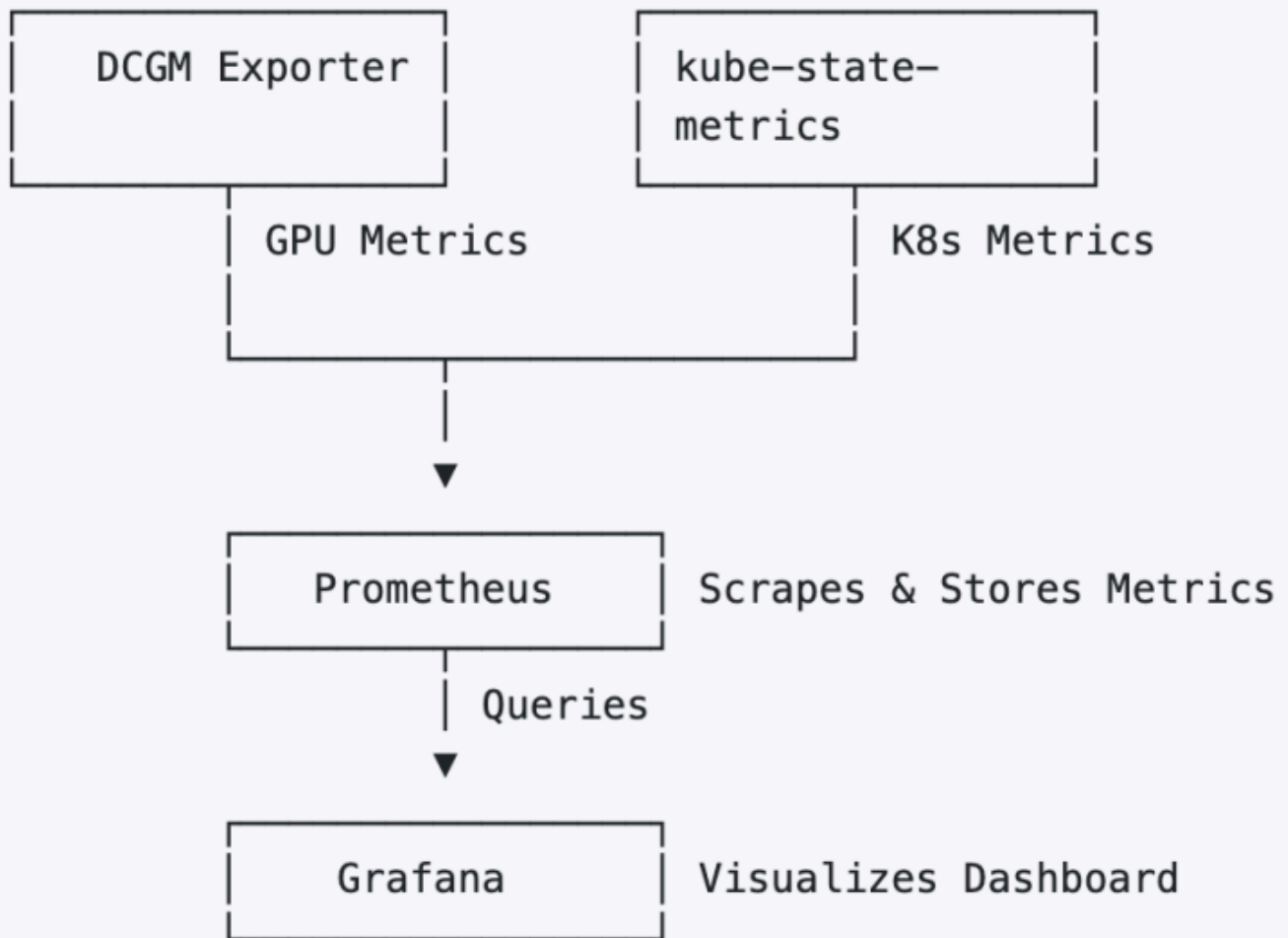
This is the section of the whole curriculum that maps most directly onto "research infrastructure / ML platform engineering" as a job description rather than "can write a training loop" — every other phase is about getting a training run to produce correct numbers; this one is about what happens when the run doesn't cooperate, at a scale where that's not a hypothetical.

§7

7. Monitoring & observability

The same three-layer pattern shows up whether you're watching a loss curve or a GPU.

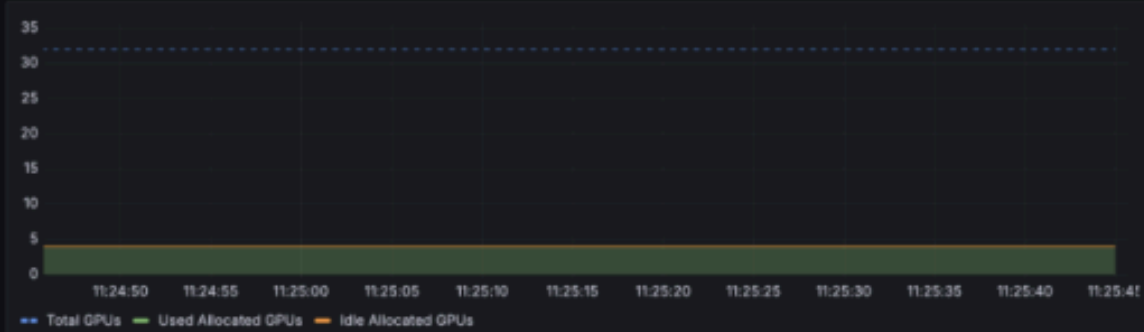
§1's `make_dashboard.py` (JSONL log → self-contained HTML dashboard) and the production GPU-monitoring stack are the same architecture, one layer apart: **collect → store as time series → visualize**. What production adds isn't a different idea, it's scale, a second data source, and — the part that actually changes behavior — alerting.



DCGM Exporter polls NVIDIA's Data Center GPU Manager for hardware telemetry (utilization, memory, temperature, power, ECC and Xid error counts) and exposes it over HTTP in Prometheus's text format; **kube-state-metrics** exposes the Kubernetes-level view (which pod is on which GPU) alongside it. **Prometheus** scrapes both on an interval and stores the resulting time series; **Grafana** queries Prometheus to render dashboards. [NVIDIA Developer Blog, 2026](#)

Concretely, this is what comes out the other end — a real Grafana dashboard built on exactly this stack:

GPU Allocation



GPU Utilization



GPU Memory Usage



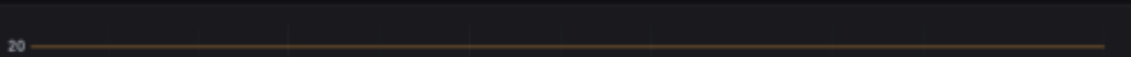
Running GPU Pods

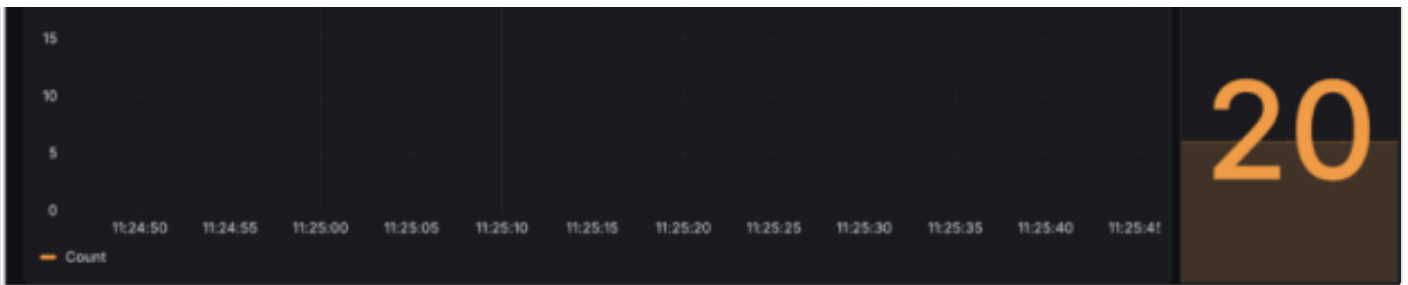


Top 10 Inefficient Pods

Pod	Namespace	Node	Utilization
runai-85dd49c72251-0-0	runai-team-d	runai-cluster-worker-0-2	92%
runai-52e74c5970c5-0-0	runai-team-d	runai-cluster-worker-0-2	94%
runai-74e101bccda4-0-0	runai-team-d	runai-cluster-worker-0-2	97%
runai-714d3fdcc465-0-0	runai-team-d	runai-cluster-worker-0-2	98%

Pending GPU Pods





GPU allocation, utilization %, memory usage, running/pending pod counts, and — the operationally useful part — a ranked table of the least-utilized pods currently holding a GPU. This is the view an on-call ML infra engineer actually works from. [NVIDIA Developer Blog, 2026](#)

Two different things get monitored, and conflating them is a mistake

Infrastructure metrics (DCGM: is the hardware healthy) and **training metrics** (loss, grad norm, LR, tokens/sec — §1's dashboard) answer different questions and usually live in different tools for that reason — Weights & Biases / MLflow for the training-run layer, Prometheus/Grafana for the fleet layer. A GPU can be 100% utilized and perfectly healthy while training has silently diverged; a training run can look numerically fine while one node's HBM is throwing correctable ECC errors that will eventually become an uncorrectable one.

Alerting is the actual behavior change, not the dashboard

A dashboard is something a human has to be looking at. Prometheus's Alertmanager evaluates rules continuously and pages on threshold — `Xid error count > 0` is often the earliest signal a GPU is degrading, well before it fails outright and takes a training job down with it. This is the mechanism that turns "job crashed, someone eventually notices, checkpoint/resume kicks in" (§6, Level 1) into pre-emptive node draining before the crash happens at all.

Loss-curve monitoring as an active control, not passive logging

The PaLM paper (Chowdhery et al., [2022](#)) documents roughly 20 loss spikes over the course of training a 540B model, of unclear cause. Their mitigation was automated, not manual: detect a spike against the monitored loss curve, roll back to a checkpoint from ~100 steps earlier, skip the batch(es) suspected of triggering it, resume. The monitoring system doesn't just observe here — it *triggers* the same checkpoint/resume mechanism from §1 and §6, automatically, mid-run. Monitoring, checkpointing, and fault tolerance aren't three separate systems in a mature stack; they're one control loop with three names.

8. Production landscape

Where the toy implementations above sit relative to what's actually run.

LAYER	SYSTEM	NOTE
training framework	TorchTitan	reference impl. of everything in §3–4 at real pretraining scale
training framework	DeepSpeed	the other major ZeRO implementation; adds NVMe offload (ZeRO-Infinity)
training framework	Megatron-Core	NVIDIA's maintained 3D-parallel library, not just the 2019 paper
communication	NCCL	what gloo stands in for — ring/tree collectives, topology-aware
control plane	Slurm / Kubeflow	job-scheduling layer above torchrun
paradigm	Ray / Ray Train	actor-based — a third paradigm alongside SPMD and single-controller
serving	vLLM / SGLang	adjacent — inference, not training (PagedAttention, continuous batching)
observability	W&B / MLflow	the experiment-tracking layer; DCGM/Prometheus is hardware, this is metrics

Full implementation, all experiments, phase-by-phase writeup:

github.com/longyi1207/ai_lab/dist_training_lab